

枚举综合教学文档

一、教学目标

二、基础案例：快递状态

1. 定义枚举
2. 快递控制器
3. 效果

三、进阶案例：饮料机接口

1. 定义枚举（带价格）
2. 通用返回对象
3. 控制器
4. 效果

一、教学目标

1. 理解 枚举（Enum） 在项目中的作用与使用场景。
2. 掌握 自定义返回对象 与 统一 JSON 接口风格。
3. 熟悉 枚举与接口结合 的实际应用，如快递状态、饮料机菜单。

二、基础案例：快递状态

1. 定义枚举

```
1 public enum ExpressStatus {  
2     CREATED("已揽收"), TRANSIT("在途中"), SUCCESS("签收");  
3  
4     private final String label;  
5  
6     ExpressStatus(String label) {  
7         this.label = label;  
8     }  
9  
10    public String getLabel() {  
11        return label;  
12    }  
13 }
```

2. 快递控制器

```
1  @RestController
2  @RequestMapping("/express")
3  public class ExpressController {
4
5      /**
6       * 获取快递状态
7       *
8       * @param status 快递状态
9       * @return String
10      */
11      @GetMapping("/{status}")
12      public String checkExpressStatus(@PathVariable ExpressStatus status) {
13          return "快递状态: " + status.getLabel();
14      }
15
16      /**
17       * 获取所有快递状态
18       *
19       * @return List<Map < String, String>>
20      */
21      @GetMapping("/express/statuses")
22      public List<Map<String, String>> getAllStatuses() {
23          List<Map<String, String>> list = new ArrayList<>();
24          for (ExpressStatus status : ExpressStatus.values()) {
25              Map<String, String> item = new HashMap<>();
26              item.put("value", status.name());
27              item.put("label", status.getLabel());
28              list.add(item);
29          }
30          return list;
31      }
32
33      /**
34       * 更新快递状态
35       *
36       * @param orderNo 快递单号
37       * @param status 快递状态
38       * @return String
39      */
40      @PostMapping("/express/update")
41      public String updateExpress(@RequestParam String orderNo, @RequestParam ExpressStatus status) {
42          // 实际应用会更新数据库
43          return "快递单号 " + orderNo + " 状态已更新为: " + status.getLabel();
44      }
45  }
```

```
44     }  
45 }  
46
```

3. 效果

- GET /express/CREATED → 快递状态：已揽收

The screenshot shows a REST client interface with the following details:

- Request:** GET http://localhost:8888 /express/{status}
- Params:** One path parameter 'status' with value 'CREATED' and type 'string'.
- Response:** Status 200, 7ms, 30B. The body is '当前快递状态：已揽收'.
- Validation:** A message indicates '返回的数据格式应当是 JSON' (The returned data format should be JSON).

参数名	参数值	类型	说明
status	CREATED	string	CREATED :CREATED

参数名	参数值	类型	说明
status	CREATED	string	CREATED :CREATED

- GET /express/statuses → 返回 JSON 列表

GET 获取所有快递状态 × + ... 开 开发环境

请求 响应定义 接口说明 预览文档 测试用例 Mock 获取所有快递状态

GET http://localhost:8888 /express/express/statuses 发送 保存 ...

Params Body Headers Cookies Auth 前置操作 后置操作 设置

Query 参数

参数名	参数值	类型	说明
添加参数			

Body Cookie Header 5 控制台 实际请求 200 8ms 118B 校验响应 成功 (200) 通过

Pretty Raw Preview Visualize JSON utf8

```
1 [
2   {
3     "label": "已揽收",
4     "value": "CREATED"
5   },
6   {
7     "label": "在途中",
8     "value": "TRANSIT"
9   },
10  {
11    "label": "签收",
12    "value": "SUCCESS"
13  }
14 ]
```

- POST /express/update?orderNo=123&status=RANSIT → 快递单号 123 状态已更新为：在途中

POST 更新快递状态 • + ... 开 开发环境

请求 响应定义 接口说明 预览文档 测试用例 Mock 更新快递状态

POST http://localhost:8888 /express/express/update 发送 保存 ...

Params 2 Body Headers Cookies Auth 前置操作 后置操作 设置

Query 参数

✓ 参数名	参数值	类型 *	说明
✓ orderNo	= 123	string *	快递单号 更多
✓ status	= TRANSIT	string *	快递状态 CREATED :CREATED 更多

添加参数

Body Cookie Header 5 控制台 实际请求 200 4ms 47B 校验响应 成功 (200)

Pretty Raw Preview Visualize Text utf8

1 快递单号 123 状态已更新为：在途中

校验响应结果
✖ 返回的数据格式应当是 JSON

三、进阶案例：饮料机接口

1. 定义枚举（带价格）

```
1 public enum DrinkType {  
2     COFFEE("咖啡", 12.0), TEA("奶茶", 10.0), JUICE("果汁", 8.0);  
3  
4     private final String label;  
5     private final double price;  
6  
7     DrinkType(String label, double price) {  
8         this.label = label;  
9         this.price = price;  
10    }  
11  
12    public String getLabel() {  
13        return label;  
14    }  
15  
16    public double getPrice() {  
17        return price;  
18    }  
19 }
```

2. 通用返回对象

```
1  @Getter
2  public class ApiResponse<T> {
3      private final int code;
4      private final String msg;
5      private final T data;
6
7      public ApiResponse(int code, String msg, T data) {
8          this.code = code;
9          this.msg = msg;
10         this.data = data;
11     }
12
13     public static <T> ApiResponse<T> success(String msg, T data) {
14         return new ApiResponse<>(200, msg, data);
15     }
16
17     public static <T> ApiResponse<T> error(String msg) {
18         return new ApiResponse<>(400, msg, null);
19     }
20 }
```

3. 控制器

```
1  @RestController
2  @RequestMapping("/drink")
3  public class DrinkController {
4
5
6      /**
7       * 选择饮料
8       *
9       * @param type 饮料类型
10      * @return ApiResponse
11      */
12      @GetMapping("/{type}")
13      public ApiResponse<Map<String, Object>> chooseDrink(@PathVariable String type) {
14          try {
15              // 将传入的 type 统一转为大写字符
16              DrinkType drink = DrinkType.valueOf(type.toUpperCase());
17              // 构造返回结果
18              Map<String, Object> result = new HashMap<>();
19              result.put("name", drink.getLabel());
20              result.put("price", drink.getPrice());
21              // 返回结果
22              return ApiResponse.success("选择成功", result);
23          } catch (IllegalArgumentException e) {
24              // 输入的参数无效
25              return ApiResponse.error("饮料类型无效");
26          }
27      }
28
29
30      /**
31       * 获取饮料菜单
32       *
33       * @return ApiResponse
34       */
35      @GetMapping("/menu")
36      public ApiResponse<List<Map<String, Object>>> getMenu() {
37          List<Map<String, Object>> menu = new ArrayList<>();
38          for (DrinkType drink : DrinkType.values()) {
39              Map<String, Object> item = new HashMap<>();
40              item.put("name", drink.getLabel());
41              item.put("price", drink.getPrice());
42              menu.add(item);
43          }
44      }
45  }
```



```

44         return ApiResponse.success("菜单获取成功", menu);
45     }
46
47
48     /**
49      * 下单
50      *
51      * @param type 饮料类型
52      * @param num 数量
53      * @return ApiResponse
54      */
55     @GetMapping("/order")
56     public ApiResponse<Map<String, Object>> orderDrink(@RequestParam String
57     g type, @RequestParam int num) {
58         try {
59             DrinkType drink = DrinkType.valueOf(type.toUpperCase());
60             double total = drink.getPrice() * num;
61             Map<String, Object> result = new HashMap<>();
62             result.put("name", drink.getLabel());
63             result.put("num", num);
64             result.put("unitPrice", drink.getPrice());
65             result.put("totalPrice", total);
66             return ApiResponse.success("下单成功", result);
67         } catch (IllegalArgumentException e) {
68             return ApiResponse.error("饮料类型无效");
69         }
70     }

```

4. 效果

- GET /drink/coffee → 选择饮料

GET 选择饮料 × + ... 开 开发环境

请求 响应定义 接口说明 预览文档 测试用例 Mock 选择饮料

GET http://localhost:8888 /drink/{type} 发送 保存

Params 1 Body Headers Cookies Auth 前置操作 后置操作 设置

Query 参数

参数名	参数值	类型	说明
添加参数			

Path 参数

参数名	参数值	类型	说明
type	coffee	string	

Body Cookie Header 5 控制台 实际请求 200 8ms 71B 校验响应 成功 (200) 通过

Pretty Raw Preview Visualize JSON utf8

```
1 {
2   "code": 200,
3   "msg": "选择成功",
4   "data": {
5     "price": 12.0,
6     "name": "咖啡"
7   }
8 }
```

- GET /drink/menu → 获取饮料菜单

GET 获取饮料菜单

请求 响应定义 接口说明 预览文档 测试用例 Mock 获取饮料菜单

GET http://localhost:8888 /drink/menu 发送 保存

Params Body Headers Cookies Auth 前置操作 后置操作 设置

Query 参数

参数名	参数值	类型	说明
添加参数			

Body Cookie Header 5 控制台 实际请求

200 · 4ms · 140B 校验响应 成功 (200) 通过

Pretty Raw Preview Visualize JSON utf8

```
1 {
2   "code": 200,
3   "msg": "菜单获取成功",
4   "data": [
5     {
6       "price": 12.0,
7       "name": "咖啡"
8     },
9     {
10      "price": 10.0,
11      "name": "奶茶"
12    },
13    {
14      "price": 8.0,
15      "name": "果汁"
16    }
17  ]
18 }
```

- GET /drink/order?type=tea&num=2 → 返回总价 20.0

GET 下单

开 开发环境

请求

响应定义

接口说明

预览文档

测试用例

Mock

下单

GET

http://localhost:8888 /drink/order

发送

保存

Params 2

Body

Headers

Cookies

Auth

前置操作

后置操作

设置

Query 参数

参数名	参数值	类型 *	说明
type	= tea	string * 器	饮料类型 更多
num	= 2	integer * 器	数量 更多

添加参数

Body

Cookie

Header 5

控制台

实际请求

200

10ms

101B

校验响应

成功 (200)

通过

Pretty

Raw

Preview

Visualize

JSON

utf8

```
1 {
2   "code": 200,
3   "msg": "下单成功",
4   "data": {
5     "unitPrice": 10.0,
6     "totalPrice": 20.0,
7     "num": 2,
8     "name": "奶茶"
9   }
10 }
```